# Reinforcement Learning for Reservoir Operation with OpenHydroQual as the Simulator

OpenHydroQual-RL working notes

October 31, 2026

**Abstract**

We set up a closed-loop reinforcement learning (RL) problem in which a PyTorch agent operates a reservoir that is simulated by OpenHydroQual (OHQ) through its `openhydroqual_py` Python bindings. The toy problem is a water-supply reservoir hit by random storms: each day the agent chooses a release that must balance meeting a downstream demand against downstream flooding and uncontrolled spill. We document every step: building the OHQ model from Python, verifying the simulator output against an analytical solution (which uncovered a one-step time-stamp lag in the current OHQLib build), wrapping it as a Gymnasium environment, training a Proximal Policy Optimization (PPO) agent, and comparing it with rule-based operating policies. We close with a performance analysis that motivates replacing the interpreted OHQ library with a codegen-compiled kernel.

## Contents

**1 Overview** **3**

**2 Step 1: The toy problem** **3**
2.1 System . . . 3
2.2 Stochastic forcing . . . 4
2.3 MDP formulation . . . 5

**3 Step 2: Building the OHQ model from Python** **5**
3.1 Python bindings . . . 5
3.2 Components . . . 5
3.3 One control interval . . . 6

**4 Step 3: Verifying the simulator output** **6**
4.1 Finding: one-step time-stamp lag . . . 6
4.2 Episode mass balance . . . 7

**5 Step 4: The RL algorithm (PPO)** **8**

**6 Step 5: Results** **9**
6.1 Training . . . 9
6.2 Comparison with baselines . . . 9

**7 Step 6: Performance and the case for a codegen backend** **11**

**8 Limitations and next steps** **14**

**9 How to reproduce** **14**

# 1 Overview

Figure 1 shows the software stack. The RL agent (PyTorch) talks to a Gymnasium environment through the standard `reset/step` API. The environment generates the hydrologic forcing, computes the reward and delegates the physics to a *simulator backend*. The only backend implemented so far, `OHQBackend`, rebuilds and solves an OHQ model for each one-day control interval through `openhydroqual_py`, a pybind11 module that links `libOHQLib`. The backend interface is deliberately narrow so that a codegen-compiled OHQ kernel can later replace it (Section 7).

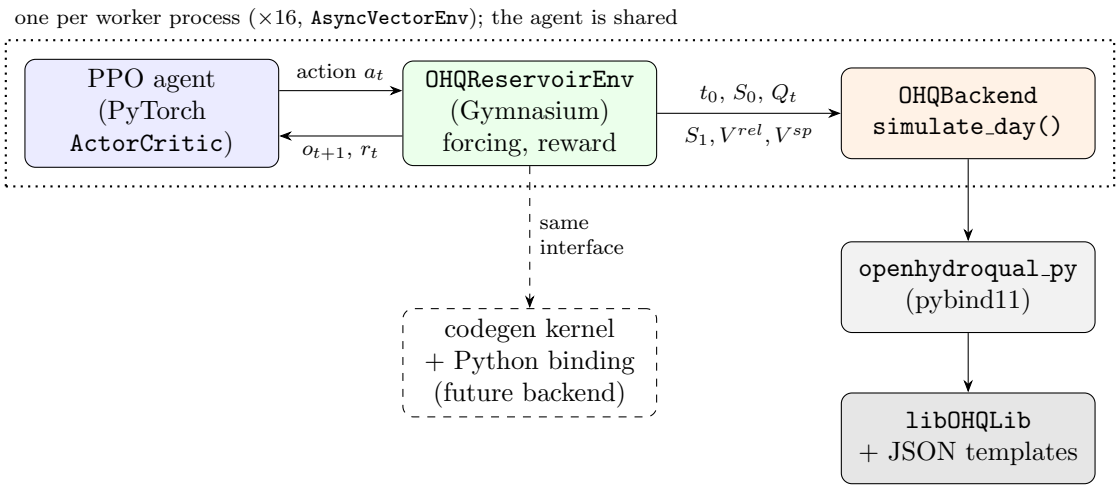



Figure 1: Software architecture. Each vectorized worker process owns one environment, one backend and its own OHQ working folder.

**Files.**

- `ohq_reservoir_env.py` – `OHQBackend`, `OHQReservoirEnv`, rule-curve baseline.
- `train_ppo.py` – PPO with GAE in plain PyTorch, parallel rollouts.
- `evaluate.py` – comparison against baselines, figures and table.
- `diagnostics.py` – simulator verification and timing figures.
- `report/` – this document.

# 2 Step 1: The toy problem

## 2.1 System

A single reservoir with spillway-crest storage $S_{\text{cap}} = 50{,}000$ m$^3$ receives a low base flow plus random storm hydrographs (Figure 2). The operator controls the release $Q_t$ through an outlet works with capacity $Q_{\max} = 6{,}000$ m$^3$/d. The downstream channel floods when its flow exceeds $Q_{\text{safe}} = 3{,}000$ m$^3$/d. Water above $S_{\text{cap}}$ leaves over an uncontrolled emergency spillway. Downstream users need a demand $D(t)$ with a mean of 1,500 m$^3$/d and a $\pm 30\%$ seasonal swing, which exceeds the mean base flow ($\approx 600$ m$^3$/d). Storage is therefore needed to bridge dry spells. Storms, on the other hand, can bring 10–35 thousand m$^3$ each.

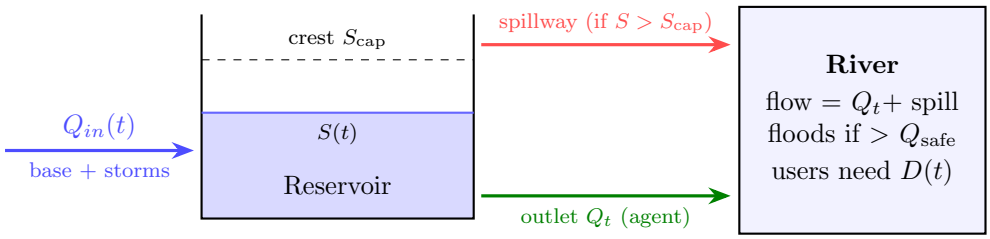



Figure 2: The toy reservoir system.

## 2.2 Stochastic forcing

For every episode (a 90-day season) the environment draws a new inflow series on a 0.05-day grid:

$$Q_{in}(t) = \beta\, q_b + \sum_j h_j(t), \qquad \beta \sim U(0.7, 1.3),\ q_b = 600\ \text{m}^3/\text{d}, \tag{1}$$

$$h_j(t) = P_j \begin{cases} \tau/t_r & 0 \le \tau < t_r \\ e^{-k_j(\tau - t_r)} & \tau \ge t_r \end{cases}, \quad \tau = t - t_j, \tag{2}$$

with storm onsets $t_j$ arriving with probability 0.06 per day, peaks $P_j \sim U(5000, 15000)$ $\text{m}^3/\text{d}$, rise time $t_r = 0.5$ d and recession constants $k_j \sim U(0.3, 0.7)$ $\text{d}^{-1}$. The demand is $D(t) = \bar{D}\,[1 + 0.3\sin(2\pi t/90 + \phi)]$ with a random phase $\phi$. The agent also receives a noisy forecast of the daily-mean inflow for the next 1, 2 and 3 days, with multiplicative log-normal errors that grow with lead time ($\sigma = 0.2\sqrt{\text{lead}}$). Figure 3 shows three realizations.

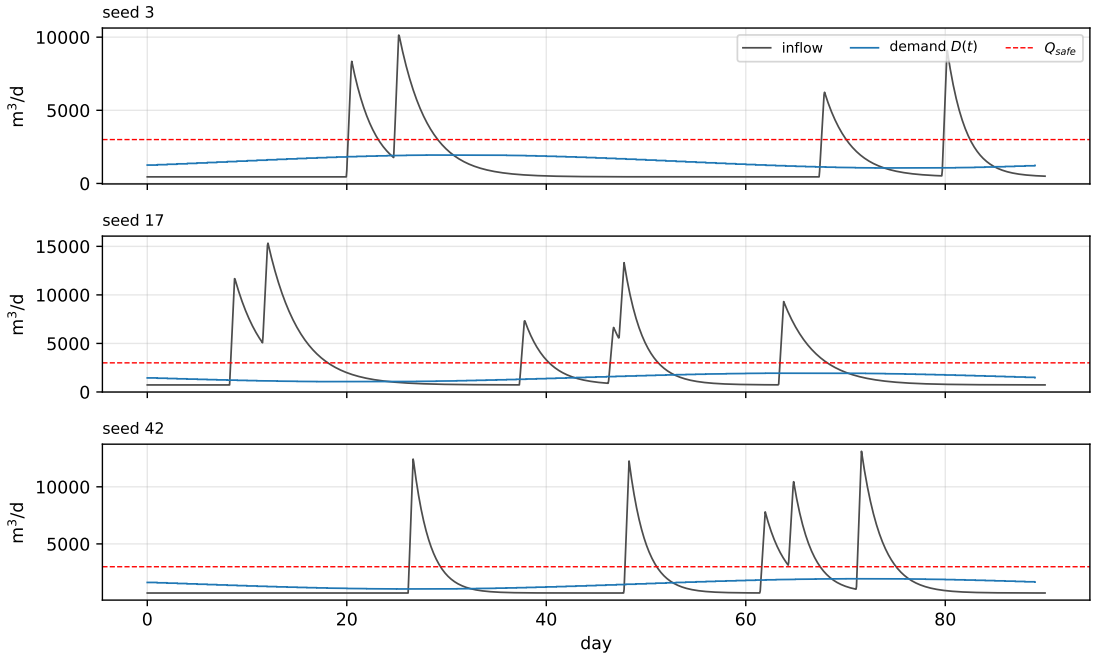



Figure 3: Three random forcing realizations (inflow, demand and the safe channel capacity).

### 2.3 MDP formulation

**Decision step:** one day; episode length 90 steps. The initial storage is drawn from $U(0.3, 0.8)\, S_{\text{cap}}$.

**Observation** (7-dim, normalized):

$$o_t = \big(S_t/S_{\text{cap}},\ Q_{in}(t)/Q_{\text{safe}},\ \hat{F}_{t+1}/Q_{\text{safe}},\ \hat{F}_{t+2}/Q_{\text{safe}},\ \hat{F}_{t+3}/Q_{\text{safe}},\ D_t/Q_{\text{safe}},\ 1 - t/90\big),$$

where $\hat{F}_{t+k}$ is the $k$-day-ahead forecast of the daily-mean inflow.

**Action:** $a_t \in [-1, 1]$, mapped to the target release $Q_t = \frac{1}{2}(a_t + 1)\, Q_{\max}$.

**Transition:** one-day OHQ simulation (Section 3).

**Reward:** with $V_t^{rel}$ and $V_t^{sp}$ the released and spilled volumes over the day,

$$r_t = -\Big(\frac{\max(0, D_t - V_t^{rel})}{D_t}\Big)^2 - w_f\, \frac{\max(0, V_t^{rel} + V_t^{sp} - Q_{\text{safe}})}{Q_{\text{safe}}} - w_s\, \frac{V_t^{sp}}{Q_{\text{safe}}}, \tag{3}$$

with $w_f = 2$ and $w_s = 1$. At the end of the season a terminal penalty $-w_T \max(0,\, 0.5 S_{\text{cap}} - S_{90})/S_{\text{cap}}$ ($w_T = 5$) discourages emptying the reservoir to avoid all flood risk.

The trade-off is the classic one: a full reservoir secures supply but leaves no room to absorb storms, while an empty one does the opposite. The release needed to make room is itself limited by $Q_{\text{safe}}$, so the agent must pre-release *before* a forecast storm arrives.

## 3 Step 2: Building the OHQ model from Python

### 3.1 Python bindings

The bindings (`OpenHydroQual/PythonBindings/bindings.cpp`) expose `System` with methods for loading templates, setting system values, creating blocks and links, `finalize`, `solve` and `write_outputs`. There is no in-memory accessor for output time series, so results are read back from the output file. Each worker therefore uses its own working folder.

### 3.2 Components

The model uses existing OHQ templates:

| OHQ object | type (template) | role |
|---|---|---|
| `Reservoir` | `Reservoir` | storage $S$, inflow time series (`Reservoir_operation.json`) |
| `River` | `fixed_head` | sink; its storage counts the released volume (`main_components.json`) |
| `SpillSink` | `fixed_head` | sink; its storage counts the spilled volume |
| `outlet` | `Reservoir_link_rule` | controlled release |
| `spillway` | `Reservoir_link_rule` | emergency overflow |

`Reservoir_link_rule` computes its flow from a piecewise-linear rule in storage:

$$Q(S) = \begin{cases} Q_{\min} & S < S_{\min} \\ Q_{\min} + \frac{S - S_{\min}}{S_{\max} - S_{\min}}(Q_{\max} - Q_{\min}) & S_{\min} \le S \le S_{\max} \\ Q_{\max} & S > S_{\max} \end{cases} \tag{4}$$

We use the same link type for both structures:

- **outlet:** $Q_{\min} = 0$, $Q_{\max} = Q_t$ (the agent's action), $S_{\min} = 0$, $S_{\max} = S_{dead} = 200$ m$^3$. The outlet releases exactly $Q_t$ unless the reservoir is nearly empty, where the release tapers to zero, so storage can never become negative.
- **spillway:** $Q_{\min} = 0$, $S_{\min} = S_{\text{cap}}$, $S_{\max} = 1.05 S_{\text{cap}}$, $Q_{\max} = 50{,}000$ m$^3$/d, which acts as an overflow weir. An earlier, steeper ramp ($1.02 S_{\text{cap}}$, $2 \times 10^5$ m$^3$/d) made the Newton solver fail to converge when the spill switched off, so the ramp was softened.

### 3.3 One control interval

Because the bindings do not expose a way to change a block's state on a live `System`, the backend rebuilds the model for every decision step, using the current storage as the initial condition. The inflow file holds the whole season in absolute time, and each step simulates $[t_0, t_0 + 1]$:

```
s = ohq.System(); s.set_default_template_path(R); s.set_working_folder(W)
s.load_template(R + "main_components.json")
s.add_template(R + "Reservoir_operation.json")
for k, v in {"simulation_start_time": t0, "simulation_end_time": t0 + 1,
             "initial_time_step": 0.05, "max_timestep_increase_factor": 1,
             "n_threads": 1, "alloutputfile": W + "output.txt"}.items():
    s.set_system_value(k, str(v))
s.create_block("Reservoir", "Reservoir", {"Storage": S0, "inflow": inflow_file})
s.create_block("River", "fixed_head", {"head": "0", "Storage": "0"})
s.create_block("SpillSink", "fixed_head", {"head": "0", "Storage": "0"})
s.create_link("outlet", "Reservoir_link_rule", "Reservoir", "River",
              {"Q_min": 0, "Q_max": Q, "S_min": 0, "S_max": S_dead})
s.create_link("spillway", "Reservoir_link_rule", "Reservoir", "SpillSink",
              {"Q_min": 0, "Q_max": 5e4, "S_min": S_cap, "S_max": 1.05 * S_cap})
s.finalize(); s.solve(True, False); s.write_outputs()
# read S(t0+1), River storage (= released volume), SpillSink storage (= spill)
```

(Numbers are converted to strings in the actual code.) Two practical notes: `set_system_value` returned `False` even when the setting was applied (fixed in OHQ, see the last section), and the `Reservoir_link_rule` template stores its `"unit"` entry inside the `rule` dictionary, so OHQ prints harmless *"property 'ft' does not exist"* messages.

## 4 Step 3: Verifying the simulator output

Before using OHQ inside an RL loop we checked it against a case with an analytical solution: constant inflow 1000 m$^3$/d, release 150 m$^3$/d and $S_0 = 1000$ m$^3$, so that $S(t) = 1000 + 850\,t$.

### 4.1 Finding: one-step time-stamp lag

The first test runs disagreed with the analytical solution, and the error grew with the time step. Reading `System::Solve` showed two causes:

1. The time loop runs `while (t < tend + dt)`, so the last step overshoots $t_{end}$. With the default adaptive step (allowed to grow to 50× the initial step), this overshoot can be large.

2. The current OHQ source stamps each output row with the time at the *end* of the step (`PopulateOutputs(true, t + dt)`), and a code comment notes this was changed. The compiled `libOHQLib.so` the bindings link against predates the change and stamps the row with the *start* time of the step. Each row therefore holds the state one step later than its label says.

Figure 4 shows the storage error relative to the analytical solution. With the step held fixed, reassigning each row the next row's stamp makes the error exactly zero. With the default growing step, the error stays offset even after this correction. The environment therefore (i) fixes the step (`max_timestep_increase_factor = 1`), (ii) reassigns each row the next row's stamp and interpolates at $t_0 + 1$, and (iii) detects automatically at start-up which convention the linked library uses, by running this analytical test. The environment keeps working if OHQLib is rebuilt with the corrected stamping.

**Update (rebuilt library).** After rebuilding `libOHQLib` (v2.1.1) and the bindings from the current source, the self-check selects the end-of-step convention. The output now matches the analytical solution exactly with both the fixed and the default growing step, and the first row holds the true initial state. The overshoot past $t_{end}$ remains by design: with the growing step, the 1-day run above ended at $t = 1.8$, so up to 80% extra time was simulated. The environment therefore keeps the step fixed. Training results are unaffected, since the physics is identical (same episode return, to 4 decimals). Per-step cost dropped from about 5.1 to 4.0 ms.

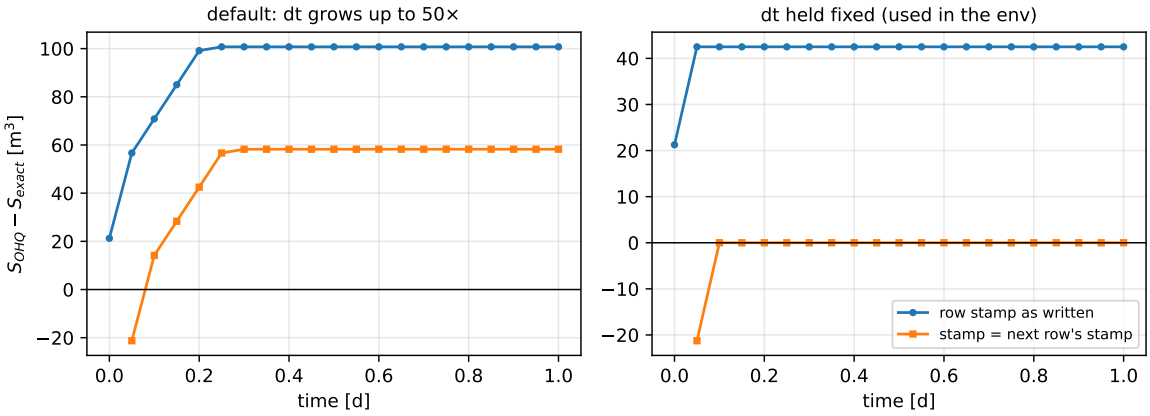



Figure 4: Error of the OHQ storage relative to the analytical solution for a one-day run (initial step 0.05 d). Left: default adaptive step. Right: fixed step, as used in the environment. Assigning each row the next row's stamp removes the error (after the very first row).

## 4.2 Episode mass balance

Over a full episode, run with the rule-curve policy, we compare the daily change in storage with the exact inflow volume (trapezoidal integral of the forcing) minus the released and spilled volumes (Figure 5). The residual follows a characteristic +/− pattern on storm days: positive on the rising limb, then negative and decaying. This is consistent with OHQ evaluating the piecewise-linear inflow series slightly offset within the implicit step, and it is a property of the time discretization, not of the bookkeeping. The residual integrates to nearly zero: the largest daily residual is 231 m$^3$, while the cumulative residual over the season is 9.3 m$^3$ out of 189,339 m$^3$ of inflow (0.005%).

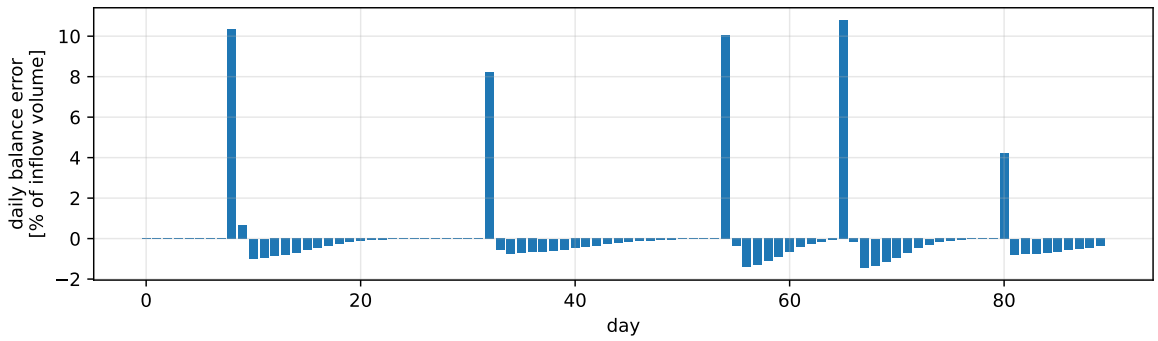



Figure 5: Daily mass-balance residual $S_t + V_t^{in} - V_t^{rel} - V_t^{sp} - S_{t+1}$ as a percentage of the day's inflow volume.

## 5 Step 4: The RL algorithm (PPO)

We use Proximal Policy Optimization [1] with generalized advantage estimation (GAE) [2], implemented directly in PyTorch (`train_ppo.py`).

**Networks.** Separate actor and critic MLPs ($7 \to 64 \to 64$, tanh activations). The policy is Gaussian, $\pi_\theta(a \mid o) = \mathcal{N}(\mu_\theta(o), \sigma^2)$, with a learned, state-independent $\log \sigma$ (initialized at $-0.5$). Sampled actions are clipped to $[-1, 1]$ by the environment. At evaluation time the deterministic action $\mathrm{clip}(\mu_\theta(o))$ is used.

**Objective.** With probability ratio $\rho_t = \pi_\theta(a_t|o_t)/\pi_{\theta_{old}}(a_t|o_t)$ and normalized advantages $\hat{A}_t$,

$$L(\theta) = -\,\mathbb{E}_t\big[\min(\rho_t \hat{A}_t,\ \mathrm{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\hat{A}_t)\big] + c_v\,\mathbb{E}_t\big[(V_\phi(o_t) - \hat{R}_t)^2\big]/2, \tag{5}$$

$$\hat{A}_t = \sum_{l \geq 0} (\gamma\lambda)^l \delta_{t+l}, \qquad \delta_t = r_t + \gamma V_\phi(o_{t+1})(1 - d_t) - V_\phi(o_t). \tag{6}$$

**Hyperparameters.**

| | | | |
|---|---|---|---|
| parallel environments | 16 | discount $\gamma$ | 0.99 |
| rollout length per env | 256 | GAE $\lambda$ | 0.95 |
| samples per update | 4096 | clip $\epsilon$ | 0.2 |
| epochs per update | 10 | value coefficient $c_v$ | 0.5 |
| minibatch size | 256 | learning rate (Adam) | $3 \times 10^{-4}$ |
| total environment steps | 409,600 | gradient-norm clip | 0.5 |

**Parallel rollouts.** Rollouts run in 16 worker processes (`gymnasium.vector.AsyncVectorEnv`, `spawn` context), each with its own OHQ instance. Gymnasium's "next-step" autoreset produces one dummy transition after every episode end. These are masked out of the update.

**A performance pitfall.** The first parallel run made almost no progress. Every worker sat at $\approx 370\%$ CPU because OHQ's solver defaults to `n_threads = 8` (OpenMP), and BLAS/PyTorch add their own threads. Sixteen workers times eight threads oversubscribed the 32-core machine. Setting `n_threads = 1` in OHQ and `OMP_NUM_THREADS = 1` / `torch.set_num_threads(1)` in the workers raised throughput to $\approx$ 1,200 environment steps per second.

# 6 Step 5: Results

## 6.1 Training

Training used 409,600 environment steps (100 PPO updates, about 4,500 episodes), which took about 5 minutes with 16 workers at ≈ 1,450 steps/s. Each step is one complete OHQ model build and solve, so this is also about $4 \times 10^5$ OHQ runs. Figure 6 shows that the mean return rises from about −52 (a near-random policy) to about −20 within the first ∼150k steps and then plateaus. The exploration standard deviation keeps shrinking (0.59 → 0.15). The remaining noise in the curve comes from the storm-to-storm variability of the episodes, not from the policy: a season with five large storms costs far more under *any* policy than a dry one.

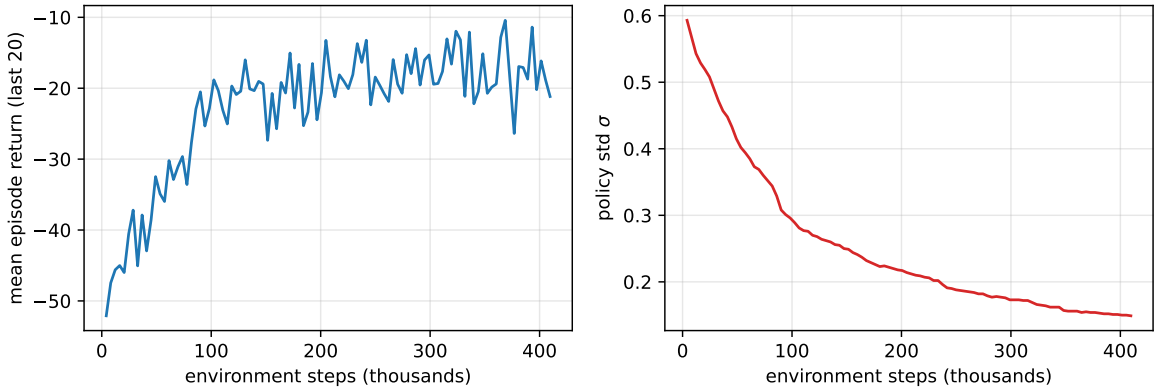



Figure 6: Left: mean return of the last 20 completed training episodes. Right: the policy's exploration standard deviation.

## 6.2 Comparison with baselines

We evaluate three policies on the same 30 held-out storm sequences (seeds 10,000–10,029, never seen in training):

**Demand only:** release $Q_t = D_t$.

**Rule curve (0.8):** a standard operating policy with a flood rule: release $D_t$, and above $\theta S_{\text{cap}}$ release $\min(Q_{\text{safe}},\ D_t + (S_t - \theta S_{\text{cap}}))$, with $\theta = 0.8$.

**Tuned rule:** the same rule with $\theta$ tuned by grid search on separate seeds (see below).

**PPO:** the trained agent (deterministic).

**Tuning the baseline.** The 0.8 threshold of the rule curve was an arbitrary first choice. To give the agent a fair opponent, we also grid-searched the threshold on 30 separate *tuning* seeds (20,000–20,029; Figure 7). The best value, a flood rule that starts releasing above 0.4 $S_{\text{cap}}$, is then evaluated on the same test seeds as everything else ("Tuned rule").

**Paired comparison.** Because every policy sees the same 30 storm sequences, the fair test is the per-episode difference in return (Table 2). The spread in Table 1 is dominated by how stormy each season is, and this pairing cancels it.

| Policy | Return | Supply | Flood | Spill | Terminal | Deficit [%] | Flood days |
|---|---|---|---|---|---|---|---|
| Demand only | $-38.41 \pm 35.47$ | -1.24 | -20.39 | -16.33 | -0.43 | 2.2 | 9.3 |
| Rule curve (0.8) | $-22.56 \pm 26.30$ | -1.31 | -13.76 | -6.88 | -0.61 | 2.3 | 6.9 |
| Tuned rule (0.4) | $-15.81 \pm 23.95$ | -2.95 | -7.73 | -3.87 | -1.25 | 5.6 | 3.5 |
| PPO | $-15.01 \pm 23.38$ | -2.88 | -7.50 | -3.39 | -1.24 | 6.0 | 6.6 |

Table 1: Mean over 30 test episodes. The return is split into its supply, flood, spill and terminal components (all penalties, so higher is better). *Deficit* is the fraction of total demand not delivered. *Flood days* counts days with river flow above $Q_{\text{safe}}$.

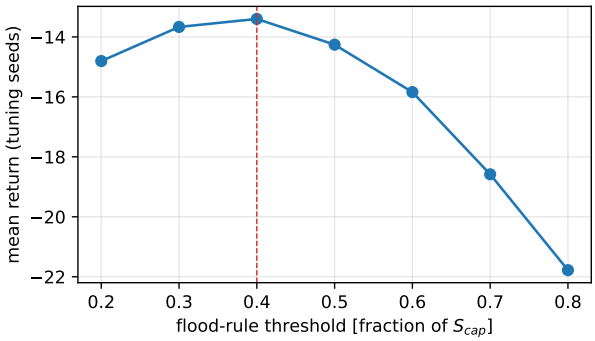



Figure 7: Mean return of the rule curve versus its flood-rule threshold, on the tuning seeds.

**What the agent learned.**

- PPO clearly beats releasing the demand only (by about 23 return points) and the untuned rule curve (by about 7.5 points, better in 20 of 30 episodes). It cuts the flood penalty roughly in half and the spill penalty by a factor of about two.
- It does this with a *drawdown hedging* strategy (Figure 9). PPO releases more than the demand most of the time and keeps the reservoir low, so storms are absorbed instead of spilled. The cost is a higher supply deficit (6% versus 2% of demand) and occasional near-empty periods. The policy map (Figure 11) shows the release increasing smoothly with both storage and the forecast inflow.
- Against the *tuned* rule curve the difference is $+0.8 \pm 0.9$, i.e. statistically indistinguishable. In effect, PPO rediscovered "keep the reservoir around 40% full and release the excess", which is close to the best policy in the rule-curve family for this reward. The two policies reach the same return differently: the tuned rule floods on fewer days, while PPO exceeds $Q_{\text{safe}}$ more often but by smaller margins.

**Interpretation.** For a single reservoir with a 3-day forecast and this reward, a well-tuned one-parameter rule is already near-optimal. RL earns its keep only when that stops being true, for example with multiple interacting reservoirs, nonlinear water-quality objectives, longer forecasts whose skill varies, or rewards that a fixed rule cannot express. The value of this exercise is the pipeline: the same environment-and-agent code applies unchanged to richer OHQ models.

| baseline | mean(PPO − baseline) | std. error | PPO better in |
|---|---|---|---|
| Demand only | +23.39 | 4.01 | 23/30 |
| Rule curve (0.8) | +7.54 | 2.11 | 20/30 |
| Tuned rule (0.4) | +0.79 | 0.86 | 12/30 |

Table 2: Paired differences in episode return, PPO minus baseline, on the 30 test episodes.

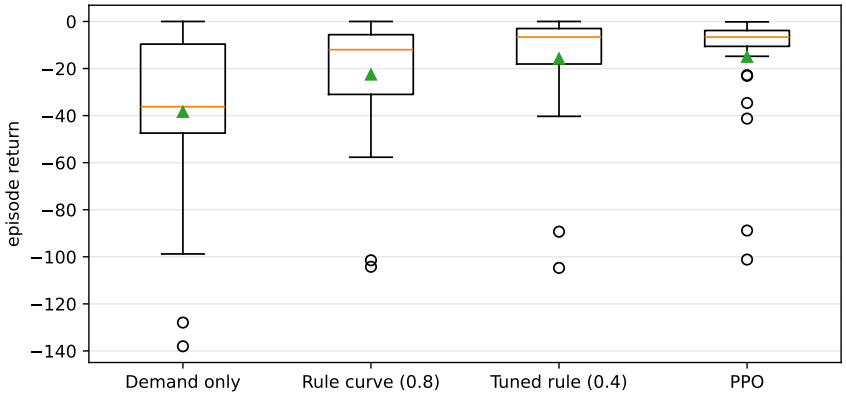



Figure 8: Distribution of test-episode returns.

## 7 Step 6: Performance and the case for a codegen backend

RL needs a very large number of short simulations; this run alone used about $4 \times 10^5$ one-day OHQ solves. Figure 12 breaks down the cost of one step.

One step costs about 4.3 ms, but the numerical solve (20 implicit steps of 0.05 d) accounts for only 0.9 ms. The remaining 3.5 ms (80%) is overhead: constructing a `System` and parsing the JSON templates (2.2 ms), building blocks and links (0.9 ms), and writing and parsing the output file (0.4 ms). The overhead is a fixed per-step cost, independent of how hard the physics is. A codegen-compiled model that is built once and only has its state reset between steps would remove nearly all of it, giving roughly a 5× speed-up for this model even before any gain in the solver itself.

**Backend contract.** Any replacement only needs to implement

```
class Backend:
    def set_inflow(self, t, q): ... # forcing for the whole episode
    def simulate_day(self, t0, S0, Q): # one control interval
        return S1, released_volume, spilled_volume
    def close(self): ...
```

and is passed to the environment as `OHQReservoirEnv(backend=...)`. A compiled kernel that keeps its state in memory between calls would also remove the per-step file round trip and the template parsing, and would allow batched, vectorized rollouts within a single process.

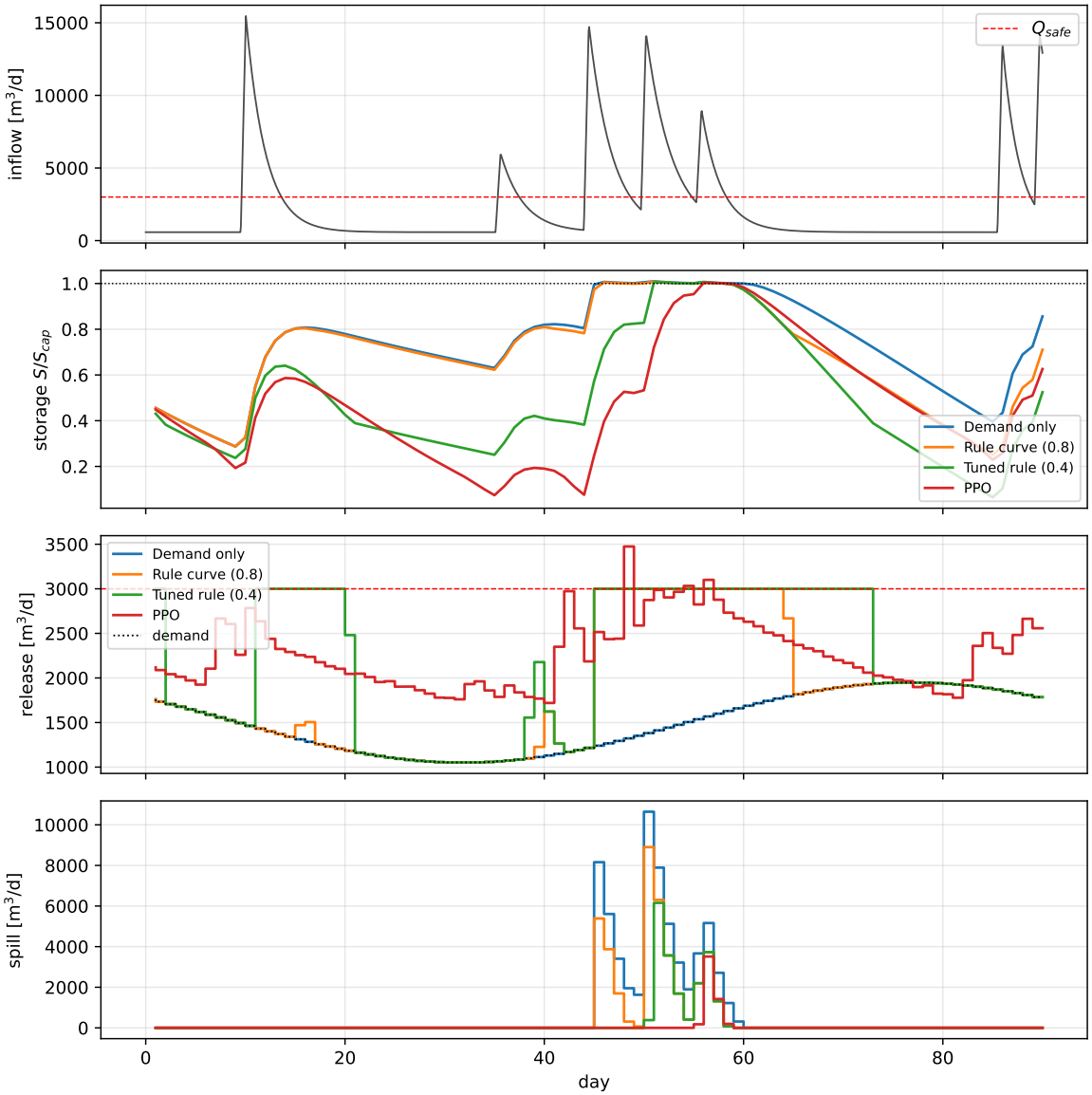



Figure 9: Test episode with the largest PPO gain over the rule curve (0.8). From top: inflow, storage, release (dotted: demand; dashed: $Q_{\text{safe}}$), spill.

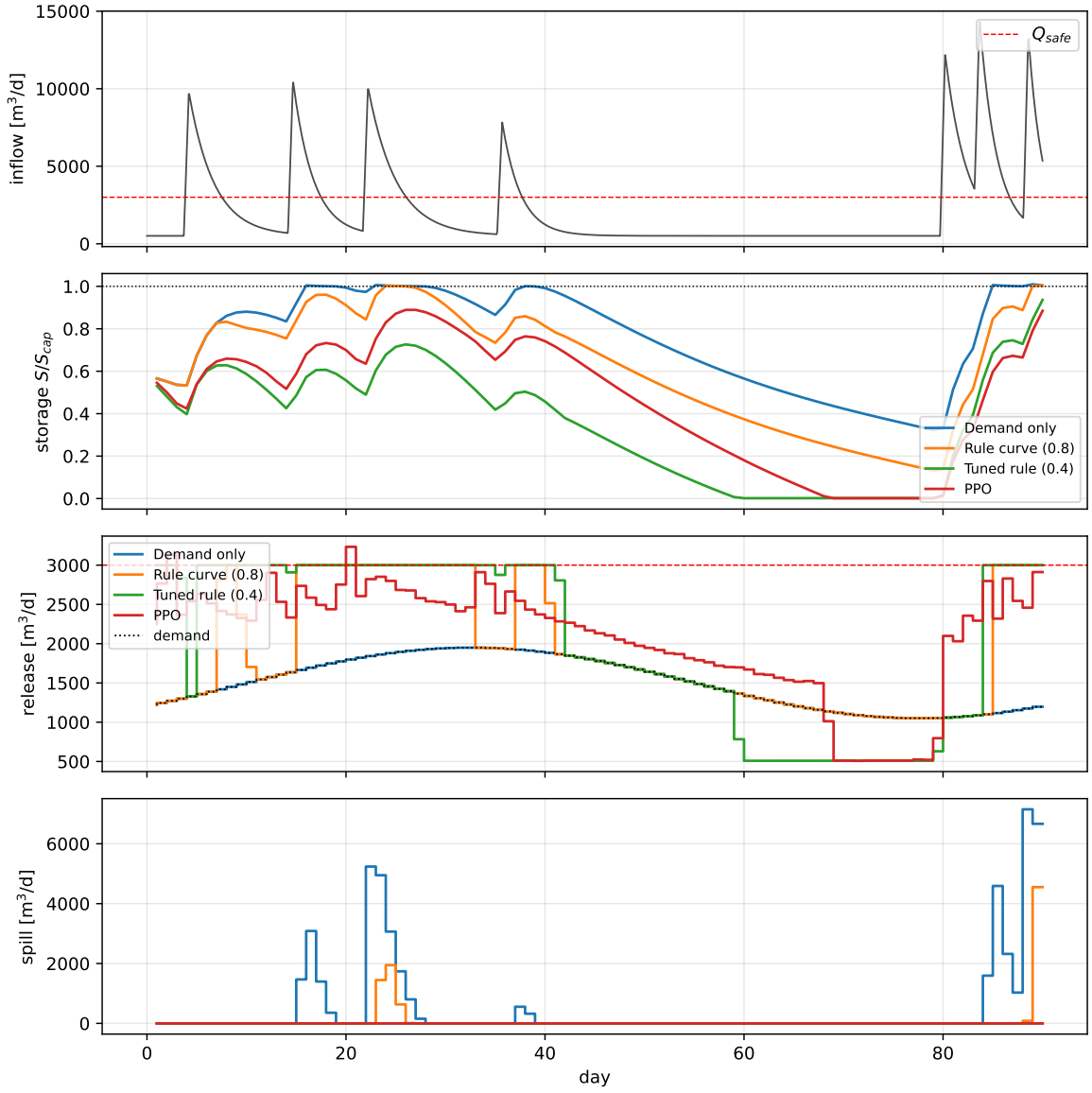



Figure 10: A test episode with the median PPO-vs-rule-curve (0.8) difference.

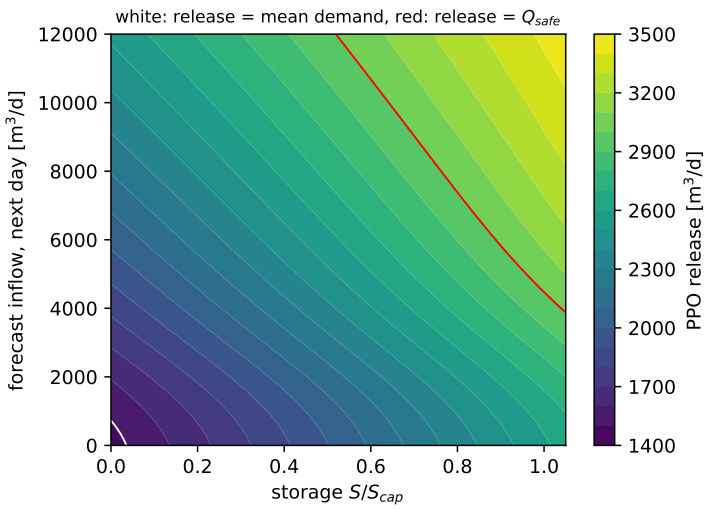



Figure 11: Learned release as a function of storage and the inflow forecast. The current inflow and the 1- and 2-day forecasts are set to the value on the vertical axis, the 3-day forecast to 0.6× that value, with mean demand at mid-season.

# 8 Limitations and next steps

- **Make the problem harder than one rule can handle.** Candidates are a cascade of reservoirs, a water-quality objective (OHQ's constituent transport), hydropower with time-varying prices, or a longer, skill-varying forecast. These are the settings where RL should pull ahead of tuned rules.
- **Simulator speed.** Replace `OHQBackend` with a codegen-compiled kernel (Section 7). Even within the current bindings, a method to reset a block's state on a persistent `System` and an in-memory output accessor would remove most of the per-step overhead.
- **OHQ fixes uncovered here.** (i) *Done:* rebuilt `libOHQLib` so the bindings use the end-of-step output stamping. (ii) *Open:* move the `"unit"` entry out of the `rule` dictionary of `Reservoir_link_rule` in `main_components.json`. (iii) *Done:* `SetSystemSettingsObjectProperties` returned `false` whenever `settings.json` had not been loaded, even though the value had been applied. It now returns `SetProperty`'s result (branch `fix/set-system-value-return`), and the environment raises an error if OHQ rejects a setting.
- **RL details.** Training used a single seed and a fixed budget. Multiple seeds, observation and reward normalization, and a tanh-squashed policy would make the agent more robust. Daily-average flood accounting also underweights sub-daily peaks; the reward could use the peak river flow instead.

# 9 How to reproduce

```
source ~/Projects/ohq_rl_env/bin/activate
cd ~/Projects/OpenHydroQual-RL
```

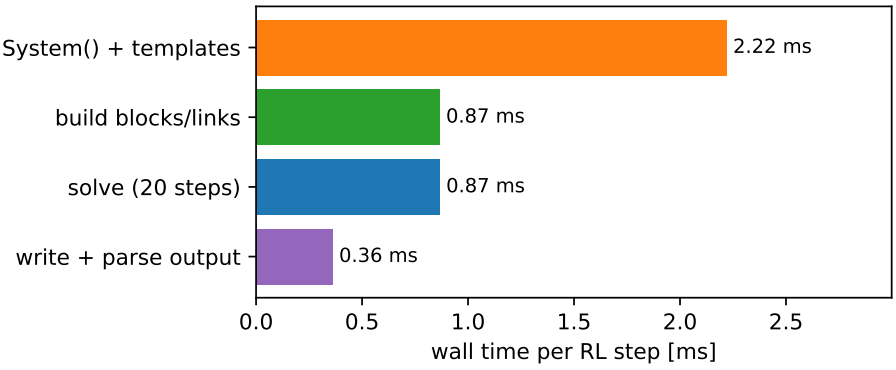



Figure 12: Wall time per environment step for the interpreted OHQ backend (mean of 50 steps, single thread).

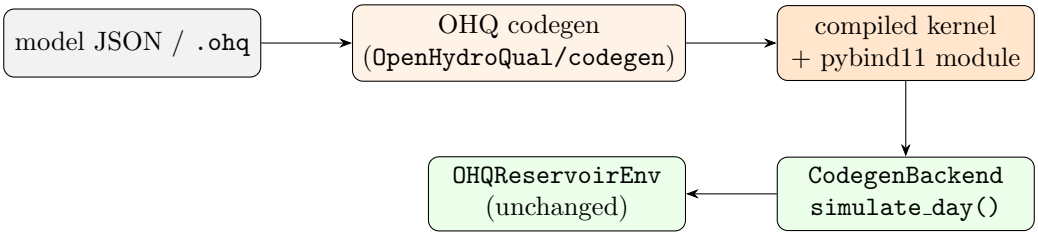



Figure 13: Planned replacement of the interpreted backend.

```
python diagnostics.py # verification + timing figures
python train_ppo.py --total-steps 409600 --num-envs 16 --out results/ppo_reservoir.pt
python evaluate.py --policy results/ppo_reservoir.pt --episodes 30
cd report && latexmk -pdf main.tex
```

The paths of the bindings and templates can be overridden with the `OHQ_PY_PATH` and `OHQ_RESOURCES` environment variables.

## References

[1] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017.

[2] J. Schulman, P. Moritz, S. Levine, M. Jordan, P. Abbeel. High-Dimensional Continuous Control Using Generalized Advantage Estimation. ICLR, 2016.